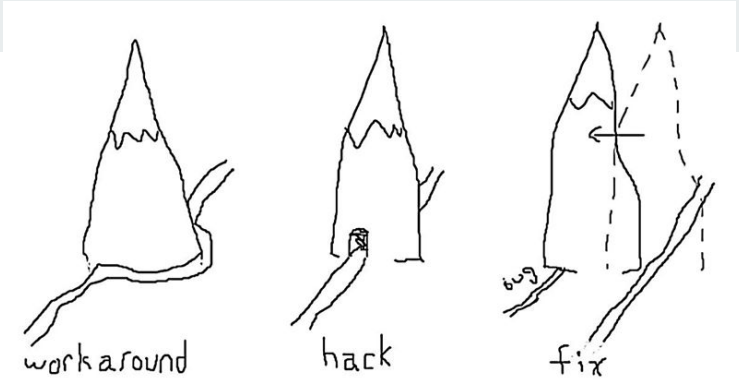
TP 0: Cuartetos de primos y reingeniería



TP 0 - Reingeniería

- No modificar la firma de las funciones.
- No cambiar el resultado de las funciones.
- No cambiar el entorno.
- No corregir bugs.
- Sí, dije NO CORREGIR BUGS (Pero reportarlos)

- Mantenerlo SIMPLE.





TP 0

Entender el algoritmo:

- Buscar divisores hasta sqrt(n)
  - Hasta n**0,5
  - Hasta que i*i >n
- Al primer divisor, devolver False.
- Buscar sólo divisores impares (OJO)
- Buscar sólo divisores primos ⇒ Criba de Eratóstenes
- Avanzar n de 10 en 10
  - De 30 en 30. Pasos de 210. O de 2310.
  - y salteo dividir por números menores que 13

TP 0

Entender la implementación:

- Un sólo print al final
- print con formato:

```
for i in range(11, 1000000, 6):
    if esprimo(i) and esprimo(i + 2) and esprimo(i + 6) and esprimo(i + 8):
        print(f"Cuarteto {contador_cuarteto}: {i}, {i + 2}, {i + 6}, {i + 8}")
        contador_cuarteto += 1
```

TP 0 - Conclusiones

+1

10º) Pérez, G. y Bonilla,, V.	2,43	5º) Barberis, Juan	0,210
9º) Marca, Pamela	2,39	4º) Vázquez, Rodrigo	0,010
8º) Llabres, Ezequiel	2,30	3º) Medela, Mariano	0,007
7º) Billordo, Axel	2,23	2º) Monges, Rafael	0,006
6º) Superno, Lautaro	1,06	1º) Huzan, Hugo	0,002

TP 0

CuartetosPrimos.py | TP0_Huzan_67910.py *

```python
import time
def cribar(n):
    # Criba de Eratóstenes (Recuerdo de un TP de "Sistemas Operativos")
    # criba[i] representa al número impar 2*i + 1
    criba = bytearray(b'\x01') * (n // 2 + 1)
    criba[0] = 0  # 1 no es primo
    limite = int(n ** 0.5)
    for p in range(3, limite + 1, 2):
        if criba[p // 2]:
            inicio = p * p // 2
            cantidad = (len(criba) - 1 - inicio) // p + 1
            criba[inicio::p] = b'\x00' * cantidad
    return criba

def prog(N=1_000_000):
    t1 = time.time()
    primo = cribar(N)
    cuartetos = []
    for base in range(11, N - 8, 210):
        i = base // 2  # Se divide por 2 porque el array "primo" contiene solo impares
        if primo[i] and primo[i + 1] and primo[i + 3] and primo[i + 4]:
            cuartetos.append(f"{base} {base + 2} {base + 6} {base + 8}")
        i += 45  # Equivale a sumar 90 a la base y luego dividir por 2
        if base + 98 < N and primo[i] and primo[i + 1] and primo[i + 3] and primo[i + 4]:
            cuartetos.append(f"{base + 90} {base + 92} {base + 96} {base + 98}")
        i += 45
        if base + 188 < N and primo[i] and primo[i + 1] and primo[i + 3] and primo[i + 4]:
            cuartetos.append(f"{base + 180} {base + 182} {base + 186} {base + 188}")
    print("\n".join(cuartetos))
    t2 = time.time()
    print(t2 - t1)

prog()
```

- Criba
- en un bytearray
- de a 210
- un sólo print
- y con formato

⇒ 0,002 seg.

TP 0 - Números primos

Algoritmo para ver si un número es primo:

- Este algoritmo es O(sqrt(n))
- Siendo B la cantidad de bits de n (el tamaño de la entrada, B= lg(n)
- Entonces el algoritmo es O(2^(B/2))

- Si crece la cantidad de bits, el tiempo aumenta exponenciamente.

primos.py

```python
import time
from math import sqrt

def esprimo(n):
    if n%2==0:
        return False
    i = 3
    r=sqrt(n)
    while i<=r:
        if n%i == 0:
            return False
        i = i+2
    return True

```

Números primos

Algoritmo para ver si un número es primo:

- Hay un teorema de Fermat (no es EL teorema de Fermat) que dice: Si n es primo, entonces:

  $a^{(n-1)} \bmod n = 1 \qquad \forall a: \{1, \ldots, n-1\}$

- y se comprobó empíricamente lo siguiente:

  si $2^{(n-1)} \bmod n = 1$, entonces n **casi** siempre es primo.

- digamos que es un “pseudoprimo”

Números primos

¿Qué tan casi?

```
14
15  def prog():
16      for i in range (2,10000):
17          a=2
18          res = a**(i-1)%i
19
20          if not(esprimo(i)) and res == 1:
21              print(i,",",end="")
22
23  prog()
```

Shell

```
>>> %Run primos.py

  341 ,561 ,645 ,1105 ,1387 ,1729 ,1905 ,2047 ,2465 ,2701 ,2821 ,3277 ,4033 ,4369 ,4371 ,4681
  ,5461 ,6601 ,7957 ,8321 ,8481 ,8911 ,

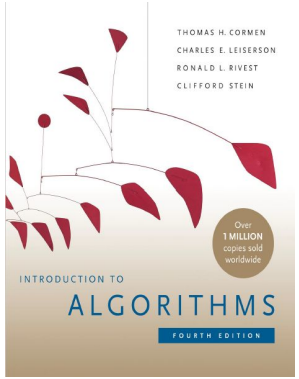
>>>
```

Números primos

- Este algoritmo falla en 22 números menores que 10.000
- La cantidad de números primos menores que n es aproximadamente n/ln(n)
- para 10000, esto es aproximadamente 1.100 primos. Este algoritmo detecta los 1100, pero se le “cuelan” 22 más.
- A medida que aumenta el tamaño de n, la probabilidad de falsos primos tiende a 0.
- Si estoy buscando un número primo muy grande aleatoriamente, el resultado es muy bueno.
- ¿Para qué buscaría un número primo aleatorio muy grande?

Números primos

- Estos números primos grandes (por ejemplo, 1024 bits) se utilizan para encriptación. (1024 bits son números de 300 cifras)
- Defino p y q dos números primos grandes. Calculo n= p. q
- Si tengo n y p ó q, es fácil hallar el otro
- Si sólo tengo n, es muy difícil hallar p y q

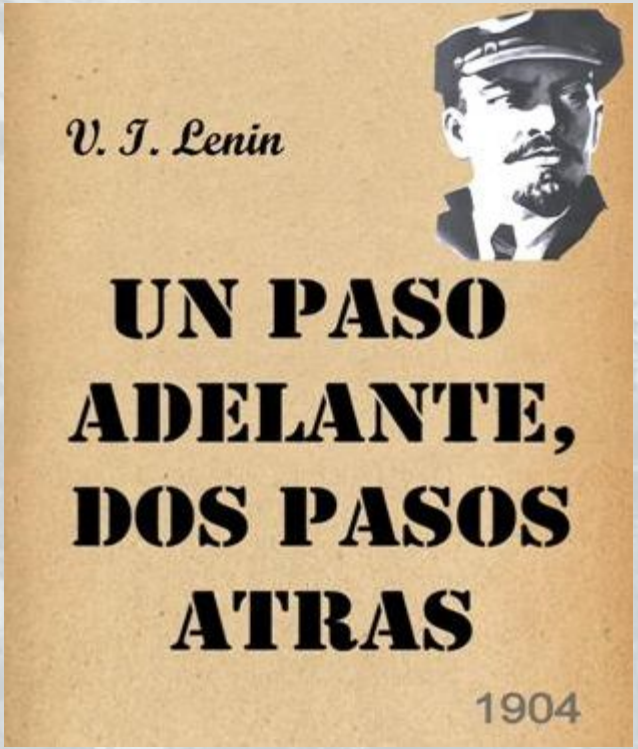


→ Fundamentos matemáticos de la criptografía.

Cap 31

Backtracking

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría



Fuerza Bruta



Créditos: Cottonbro Studio

Fuerza Bruta

¿De qué se trata?

- Buscar la solución a un problema combinatorio explorando (y analizando) todas las soluciones existentes

Es la forma más simple de resolver un problema, pero también es la más ineficiente

Fuerza Bruta (cont.)

¿Cómo se resolvería por Fuerza bruta…

- … el Problema del Viajante?
  - plantearíamos todas las posibles combinaciones de ciclos
  - calcularíamos el peso para cada uno
  - seleccionaríamos el de menor peso
- … el Problema de la Mochila?
  - plantearíamos todas las posibles combinaciones
  - descartaríamos las que exceden el peso máximo
  - seleccionaríamos la que maximiza el valor total
- … el Problema de Interval Scheduling?
  - plantearíamos todas las combinaciones de tareas
  - descartaríamos las que tienen solapamiento
  - seleccionaríamos la que tiene mayor cantidad de tareas

Fuerza Bruta (cont.)

Un algoritmo de fuerza bruta tiene más o menos esta forma:

1. Repetir:
    a. Generar la próxima solución
    b. Probar si la solución es válida
2. Mientras la solución sea inválida

---

1. Repetir:
    a. Generar la próxima solución
    b. Probar si la solución es válida
    c. Si es la mejor solución hasta ahora, guardarla
2. Mientras queden soluciones sin probar

Fuerza Bruta (cont.)

- Debe asegurarse de generar todas las soluciones posibles al problema.
- Funciona bien mientras el problema tenga una cantidad finita de soluciones.
- Pero es terriblemente lento.

Send more money

SendMoreMoneyFB.py

```
import time

def money():
    t1=time.time()
    for s in range (0,10):
        for e in range (0,10):
            for n in range (0,10):
                for d in range (0,10):
                    for m in range (0,10):
                        for o in range (0,10):
                            for r in range (0,10):
                                for y in range (0,10):
                                    send = s*1000+e*100+n*10+d
                                    more = m*1000+o*100+r*10+e
                                    money = m*10000+o*1000+n*100+e*10+y
                                    if (y!=s and y!=e and y!=n and y!=d and y!=m and y!=o and y!=r and r!=o and r!=m and
                                        r!=d and r!=n and r!=e and r!=s and o!=m and o!=d and o!=n and o!=e and o!=s and
                                        m!=d and m!=n and m!=e and m!=s and d!=n and d!=e and d!=s and n!=e and n!=s and
                                        e!=s and m!=0 and s!=0):
                                        if send + more == money:
                                            print("SEND =", send)
                                            print("MORE =", more)
                                            print("MONEY =", money)
    t2=time.time()
    print(t2-t1)
money()
```

Send more money

- ¿Cómo podríamos evitar probar todas las combinaciones?
- ¿Qué combinaciones deberíamos evitar?
  - Las que asignan la misma cifra a dos letras diferentes
  - Las que asignan 0 a S ó a M

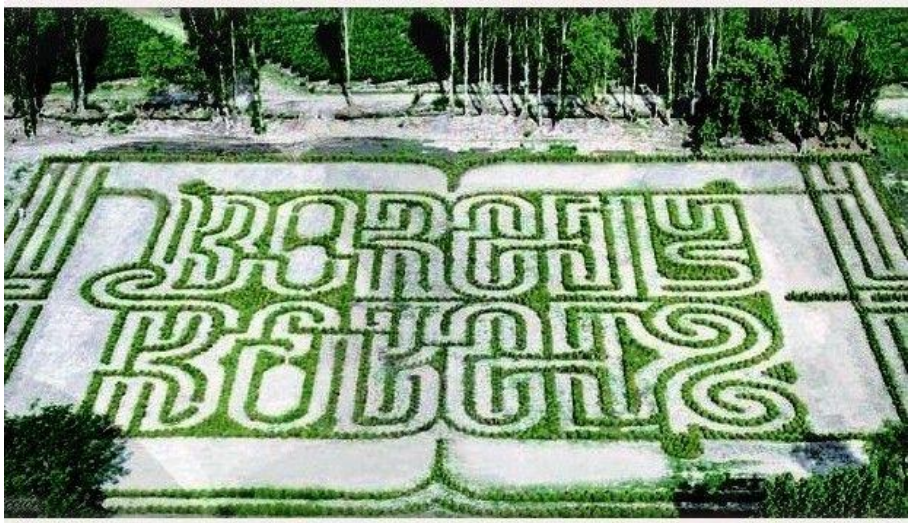
Send more money

SendMoreMoney.py

```
def money():
    t1=time.time()
    for s in range (1,10):
        for e in range (0,10):
            if e==s:
                continue
            for n in range (0,10):
                if n==e or n==s:
                    continue
                for d in range (0,10):
                    if d==n or d==e or d==s:
                        continue
                    for m in range (1,10):
                        if m==s or m==e or m==n or m==d:
                            continue
                        for o in range (0,10):
                            if o==s or o==e or o==n or o==d or o==m:
                                continue
                            for r in range (0,10):
                                if r==s or r==e or r==n or r==d or r==m or r==o:
                                    continue
                                for y in range (0,10):
                                    if y==s or y==e or y==n or y==d or y==m or y==o or y==r:
                                        continue
                                    send = s*1000+e*100+n*10+d
                                    more = m*1000+o*100+r*10+e
                                    money = m*10000+o*1000+n*100+e*10+y
                                    if send + more == money:
                                        print("SEND =", send)
                                        print("MORE =", more)
                                        print("MONEY =", money)
    t2=time.time()
    print(t2-t1)
```

75 seg → 1,7 seg

Backtracking



Créditos: Mariana Ojeda

Backtracking: Planteo General

- Son un refinamiento de Fuerza Bruta
- Lo mejoran asumiendo que las soluciones están representadas por vectores $(v_1, \ldots, v_m)$ de valores
- Igual que Fuerza Bruta, recorre el dominio de los valores hasta encontrar la solución.
- A diferencia de Fuerza Bruta, descarta una solución parcial $(v_1, \ldots, v_n)$ $(n<m)$ si ya no puede cumplir con las condiciones del problema



Cap 2

Backtracking: Planteo General

- Sólo se puede usar Backtracking en problemas tales que:
  - Permitan construir incrementalmente una solución
  - Se pueda determinar rápidamente si una solución parcial se puede completar hasta obtener una válida
- Cuando se puede usar Backtracking, se logra una mejora en velocidad frente a Fuerza Bruta (puede eliminar muchas soluciones candidatas con una sola verificación).

Backtracking: Planteo General

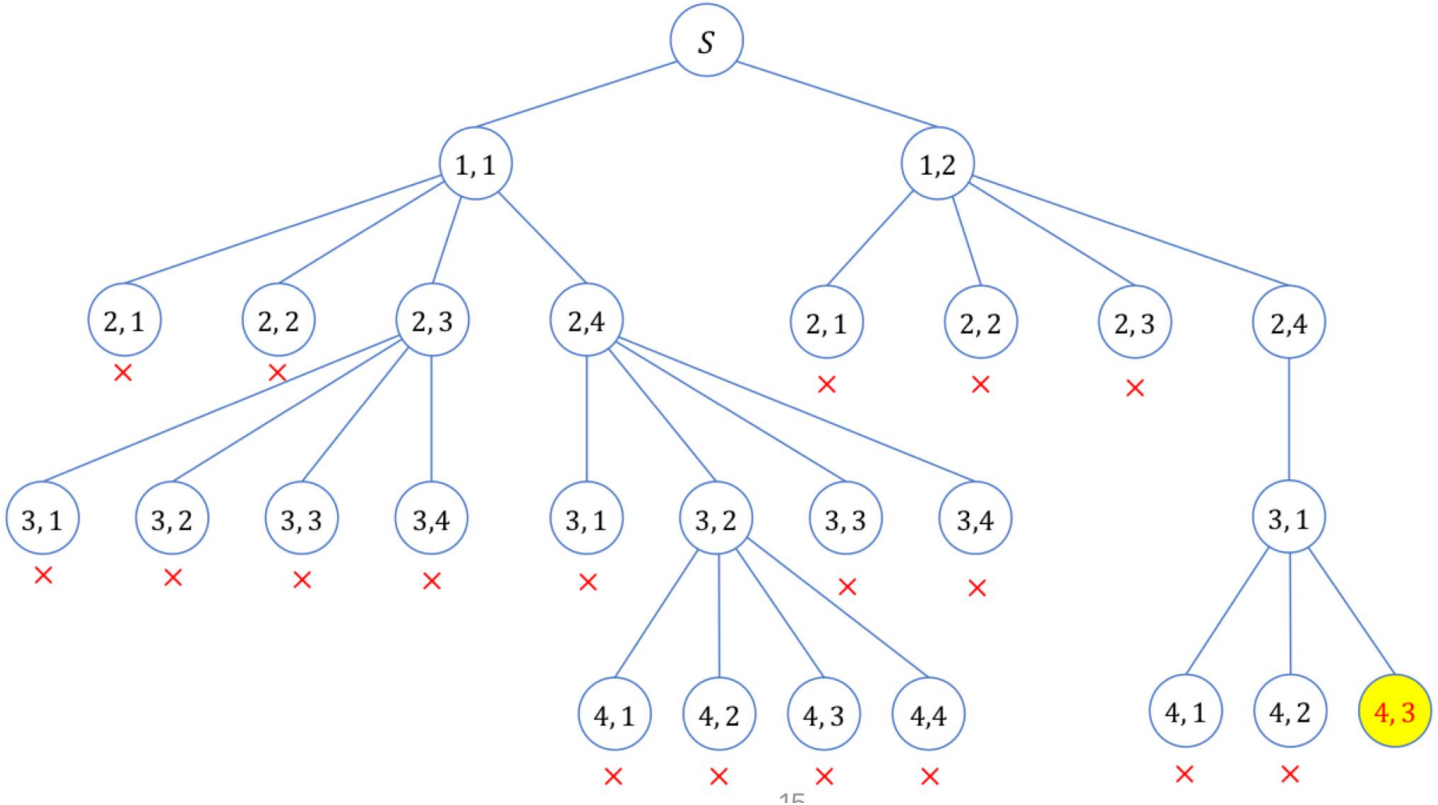


Backtracking: Árbol de Decisiones

Algunas definiciones

- Candidato Parcial: una solución incompleta
  - Se pueden representar mediante un árbol:
    - Los hijos se diferencian del padre por tener un valor más
    - Los hijos son también candidatos parciales
- Árbol de Decisión: estructura para representar a todos los candidatos parciales de un problema

Backtracking: Árbol de Decisiones (cont.)



Backtracking: Árbol de Decisiones

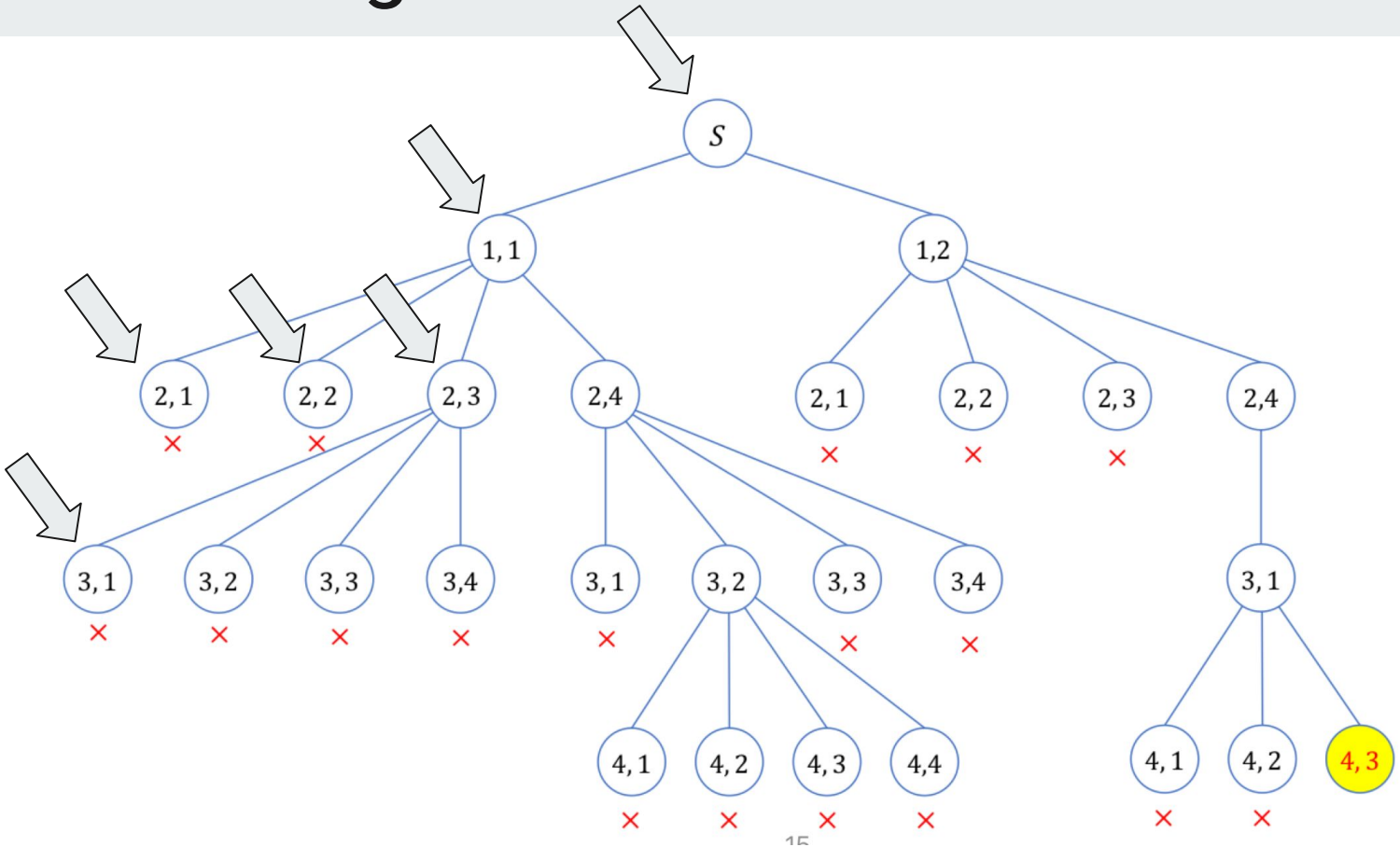
- Backtracking toma candidatos parciales y los va completando incrementalmente
  - Recorre el Árbol de Decisiones siguiendo DFS
- Podría haber muchas maneras de completar el candidato: algunas válidas, otras no.
  - Si un candidato parcial no es válido, la rama se “poda” junto con sus hijos

Backtracking: Funcionamiento

Recorre recursivamente el Árbol de Decisiones usando DFS:

1. Verifica el candidato parcial:
    a. Si es la solución:
        i. la devuelve
2. Incrementa el candidato parcial
3. Si no es válido:
    a. retrocede y vuelve a 2
4. Si es válido:
    a. recorre recursivamente desde 1 con el candidato parcial actual
5. Llegado a este punto, el problema no tiene solución

Backtracking: Árbol de Decisiones (cont.)



Problema de las N Reinas



Créditos: Cottonbro Studio

Problema de las N Reinas

- Se tiene un tablero de ajedrez de $n \; x \; n$ casillas
- Se tienen n reinas
  - Las reinas atacan en dirección horizontal, vertical y diagonal

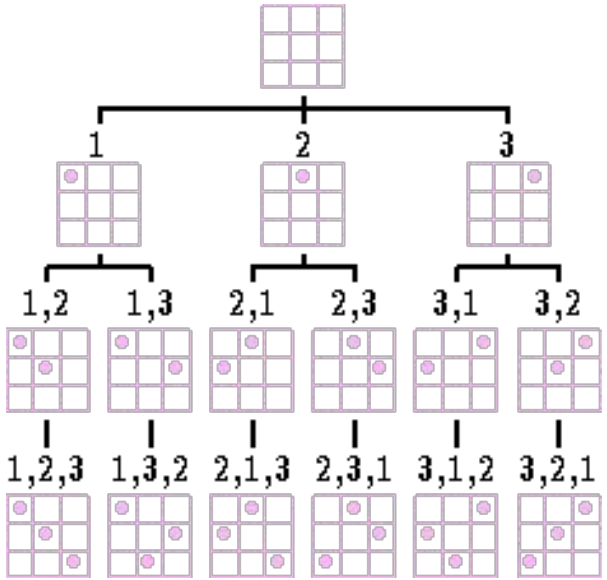
Se desean ubicar n reinas en el tablero de forma tal que ninguna pueda atacar a otra

Problema de las N Reinas: Análisis

- Una solución tendría la forma $\{e_1, e_2, .., e_n\}$, donde cada elemento e_i indica la ubicación en el tablero de la $i\text{-}ésima$ reina
- Habría n^{nxn} posibles permutaciones soluciones
- Primera poda: las reinas deben estar en ubicaciones distintas ⇒ bajamos a $n!$ soluciones posibles

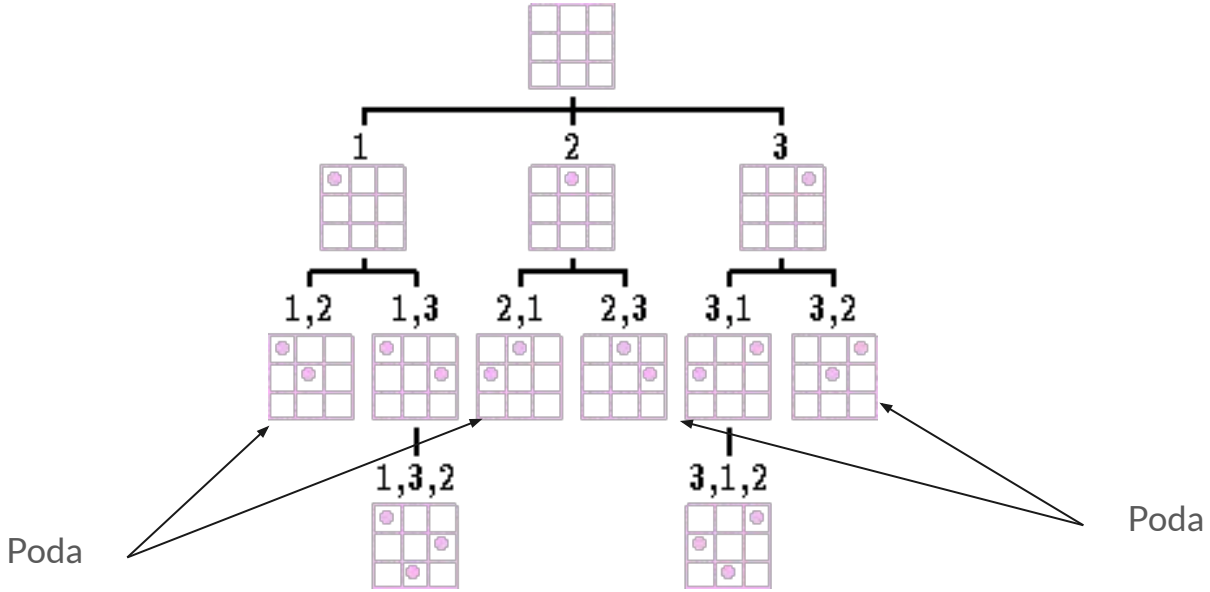
Problema de las N Reinas: Análisis

Árbol de Decisiones ($n=3$):

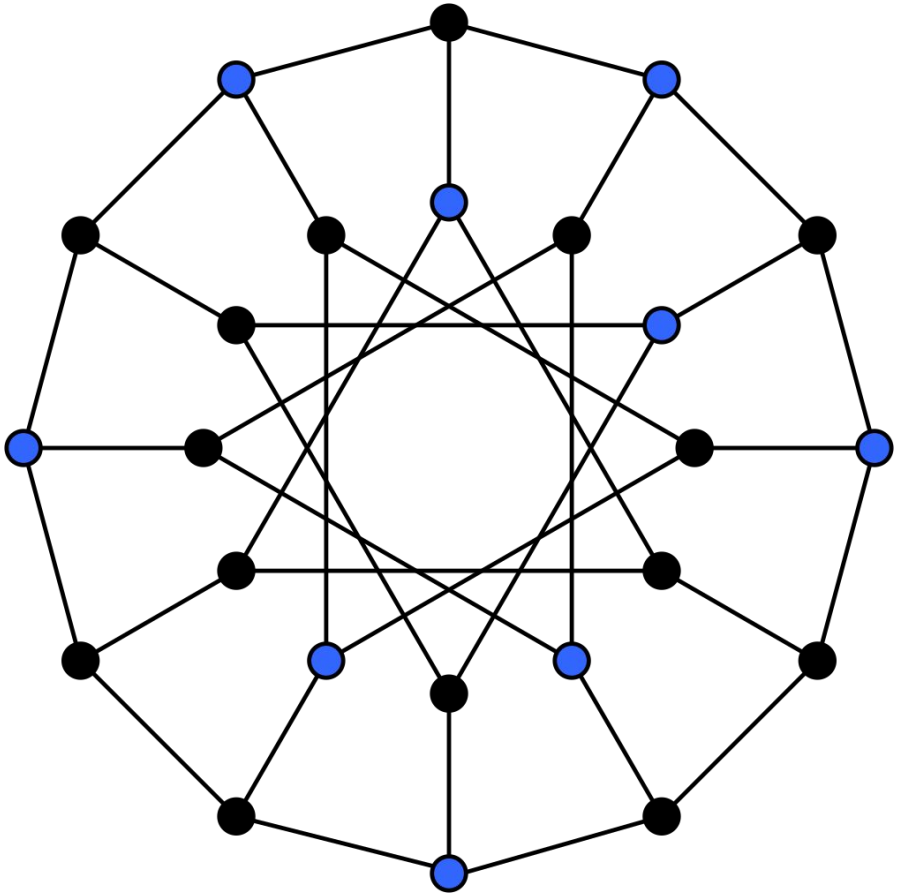


Problema de las N Reinas: Análisis

Podemos podar candidatos parciales incompatibles:



Independent Set



Créditos: Life of Riley

Independent Set: Definición

- Se tiene un grafo $G=\{V,E\}$

Un independent set S de G es un conjunto de vértices tal que para todo par de vértices $\{v_i, v_j\}$ de S no existe arista e_{ij} en E que los una

Independent Set: Ejemplo

Problema: Hallar un independent set de G de tamaño k

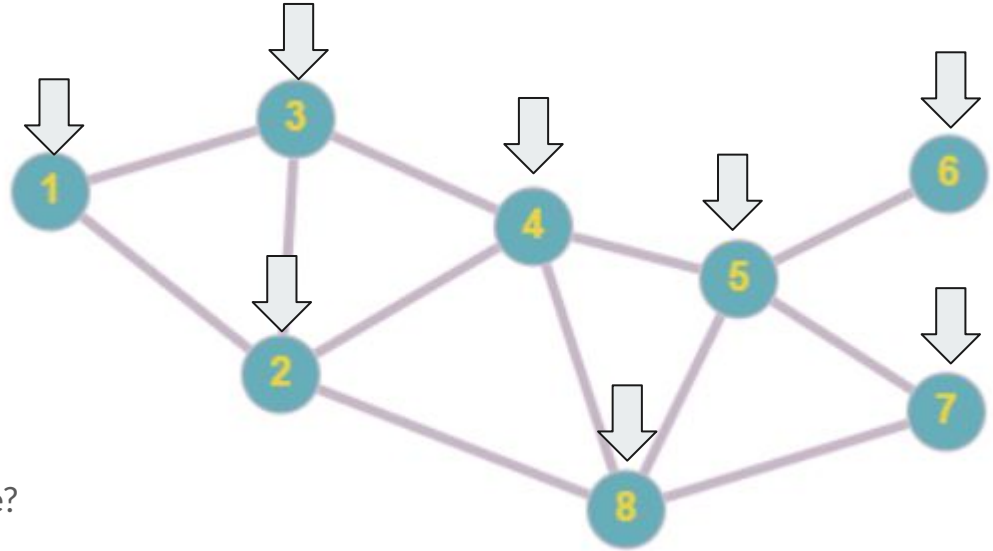
- Tenemos un grafo $G=\{V,E\}$
- Queremos guardar k elementos en algunos vértices del grafo
- No queremos que los vértices seleccionados sean adyacentes

Análisis:

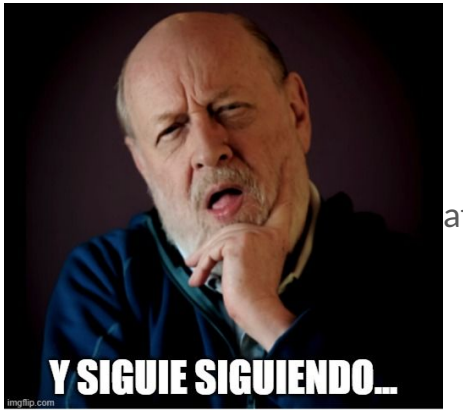
- Las soluciones tendrán la forma $\{v_1, v_2, \ldots, v_k\}$ donde v_i indica el vértice seleccionado

Independent Set: Ejemplo (cont.)

Resolvamos por Fuerza Bruta para k=4:

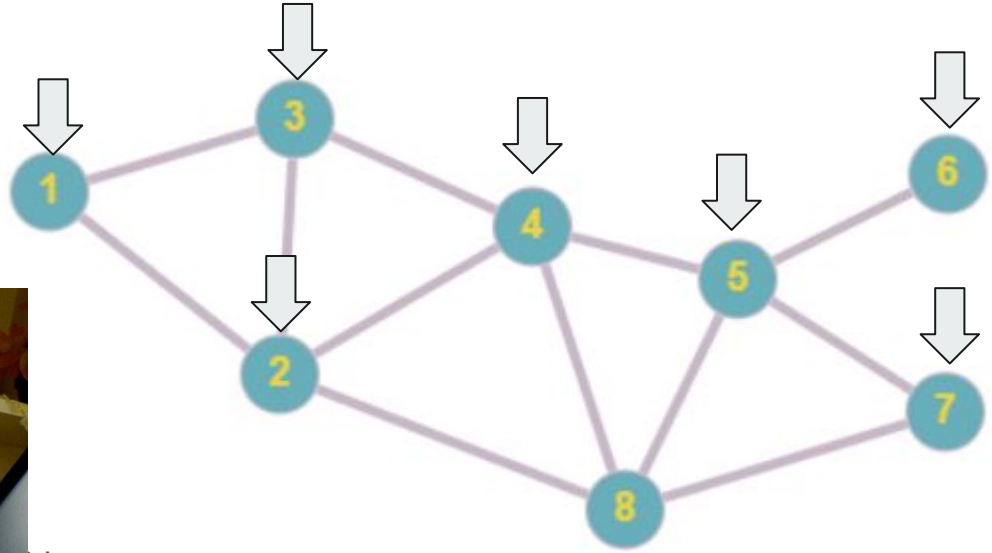


atible?

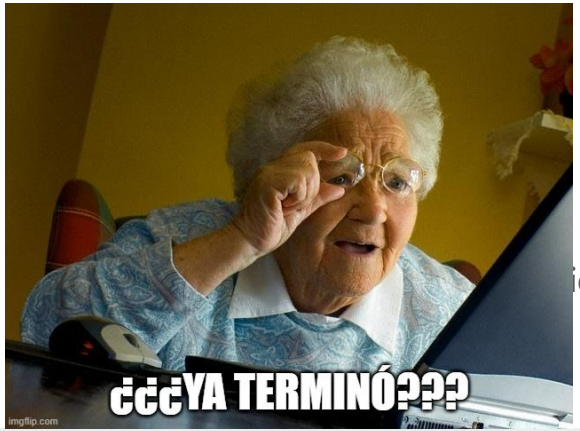


Independent Set: Ejemplo (cont.)

Resolvamos por Backtracking para k=4:



idato



Independent Set: Código Fuerza Bruta

```
# grafo: grafo a recorrer
# vertices: array con los vértices del grafo
# v_actual: índice del vértice a analizar
# puestos: lista con los vértices seleccionados
# k: cantidad de elementos a asignar
# return: true si la solución es compatible; false si no
def ubicacion_fb(grafo, vertices, v_actual, puestos, k):
  if len(puestos) == k:
    return es_compatible(grafo, puestos)
  if v_actual == len(grafo):
    return False

  #Construir próxima solución
  puestos.add(vertices[v_actual])
  if ubicacion_fb(grafo, vertices, v_actual+1, puestos, k):
    return True
  puestos.remove(vertices[v_actual])
  return ubicacion_fb(grafo, vertices, v_actual+1, puestos, k)
```

Independent Set: Código Fuerza Bruta (cont.)

```
# Version 1 - O(n^2)
# grafo: grafo a recorrer
# puestos: lista con los vértices seleccionados
def es_compatible(grafo, puestos):
  for v in puestos:
    for w in puestos:
      if v == w: continue
      if grafo.hay_arista(v,w):
        return False
  return True
```

Independent Set: Código Backtracking

```
def ubicacion_bt(grafo, vertices, v_actual, puestos, k):
  if len(puestos) == k:
    return es_compatible(grafo, puestos)
  if v_actual == len(grafo):
    return False

  # Surtido de Podas:
  if not es_compatible(grafo, puestos):
    return False
  if ya_no_llego(grafo, vertices, v_actual, puestos):
    return False

  puestos.add(vertices[v_actual])
  if ubicacion_bt(grafo, vertices, v_actual+1, puestos, k):
    return True
  puestos.remove(vertices[v_actual])
  return ubicacion_bt(grafo, vertices, v_actual+1, puestos, k)
```

Independent Set: Código Backtracking (cont.)

```
# Version 2 - O(n)
# grafo: grafo a recorrer
# puestos: lista con los vértices seleccionados
# ultimo: último vértice seleccionado
def es_compatible(grafo, puestos, ultimo_puesto):
  for w in puestos:
    if ultimo_puesto == w: continue
    if grafo.hay_arista(ultimo_puesto,w)
      return False
  return True
```

Independent Set: FB vs BT

Para tener en cuenta:

1. Fuerza Bruta:
    a. Primero construye una solución completa
    b. Después la verifica
    c. Si no es compatible, construye otra solución completa
    d. Explora TODAS las soluciones completas que existan
2. Backtracking:
    a. Construye incrementalmente la solución completa
    b. En cada incremento, verifica si el candidato parcial es compatible
    c. Si no es compatible, descarta (poda), retrocede y avanza por otro camino
    d. Si es compatible, continúa incrementando el candidato parcial

Independent Set: Complejidad

- Por Fuerza Bruta: $O(2^n)$
- Por BT: $O(2^n)$, pero la poda reduce (drásticamente) el espacio de soluciones a analizar

Independent Set: Variante

- Se tiene un grafo $G=\{V,E\}$

Encontrar todos los independent set de G que sean de tamaño k

Independent Set: Variante (cont.)

¿Cómo se modifica nuestro algoritmo?

Flashback: N Reinas

En el problema de las N Reinas:

- Si los vértices son las casillas del tablero…
- … y las aristas son las casillas que no podrían tener otra reina…

¿No estaríamos ante el problema del Independent Set?



Juegos



Créditos: Quino

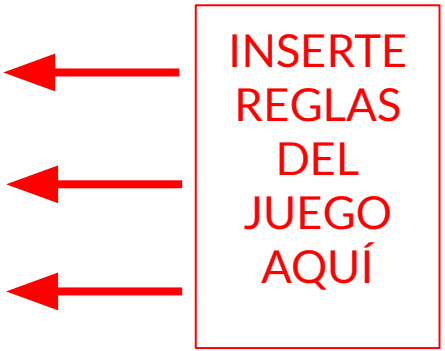
Juegos

Ernst Zermelo desarrolló un algoritmo por backtracking en **1913**, que permite ganar cualquier juego.

- Para ello definió dos estados del juego: BUENO, si en el estado actual el jugador ya ganó o tiene un camino que lo lleva a la victoria.
- El otro estado es MALO, si el jugador ya perdió o no tiene forma de evitar la derrota.
- Un jugador que ingresa a la partida en un estado BUENO tiene asegurada la victoria, aunque su oponente juegue a la perfección.
- El algoritmo recibe como datos el estado actual del tablero (X) y el jugador al que le toca mover.

Juegos

```
Jugar (X, Jugador):
    Si Jug1 ya ganó en el estado X  devolver BUENO
    Si Jug1 ya perdió en el estado X devolver MALO
    Para toda movida válida X → Y:
        Si Jugar (Y, !Jugador) == MALO
            devolver BUENO
    devolver MALO
```

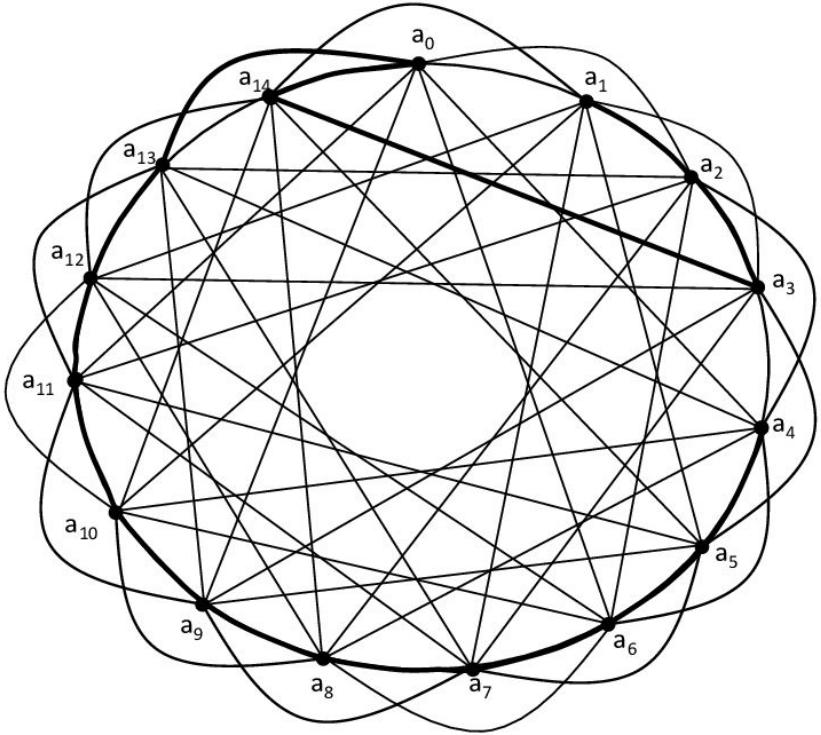


Juegos

- No interesa el pasado del juego, sólo el estado actual y el jugador que mueve.
- Contempla juegos con cantidad de movidas finita (Tatetí). Una cantidad de movidas infinita sería un empate.
- En ajedrez y otros juegos, no se pueden repetir más de tres veces una posición. ¿Cómo consideramos eso?
- En ajedrez y otros juegos, hay un límite de tiempo ¿Cómo consideramos eso?
- Este algoritmo es la base de casi todos los programas de juegos.
- Su base es “Si no pierdo, YO YA GANÉ”



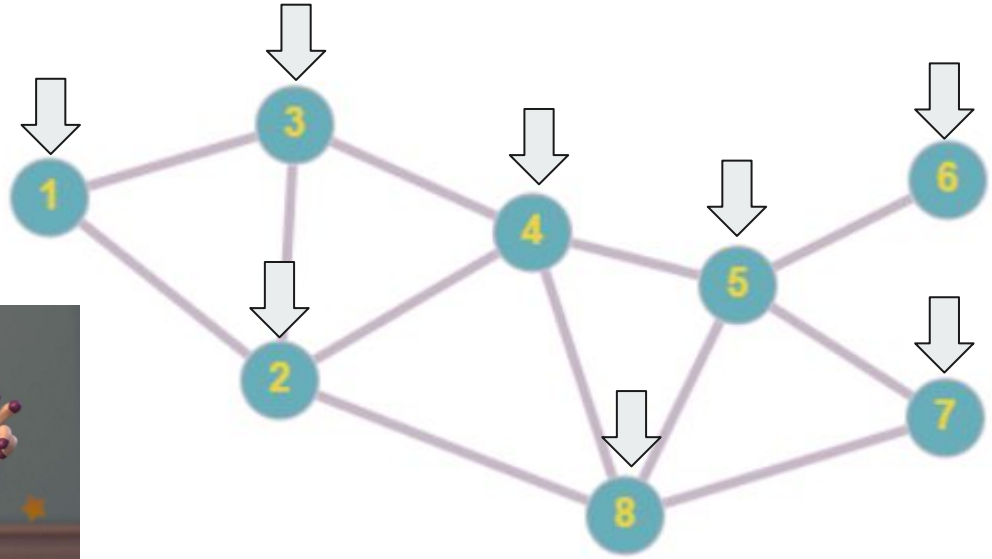
Camino Hamiltoniano



Créditos: Girisha A

Camino Hamiltoniano

Un camino Hamiltoniano recorre todos los vértices sin repetirlos





Camino Hamiltoniano: Código

```
# grafo: grafo a recorrer
# v: vértice actual
# visitados: vértices ya visitados
# camino: camino realizado
def camino_hamiltoniano_dfs(grafo, v, visitados, camino):
  visitados.add(v)
  camino.append(v)
  if len(visitados) == len(grafo)
    return True
  for w in grafo.adyacentes(v):
    if w not in visitados: # La poda
      if camino_hamiltoniano_dfs(grafo, w, visitados, camino):
        return True
  visitados.remove(v) # permitir volver por otro camino
  camino.pop()
  return False
```

DFS

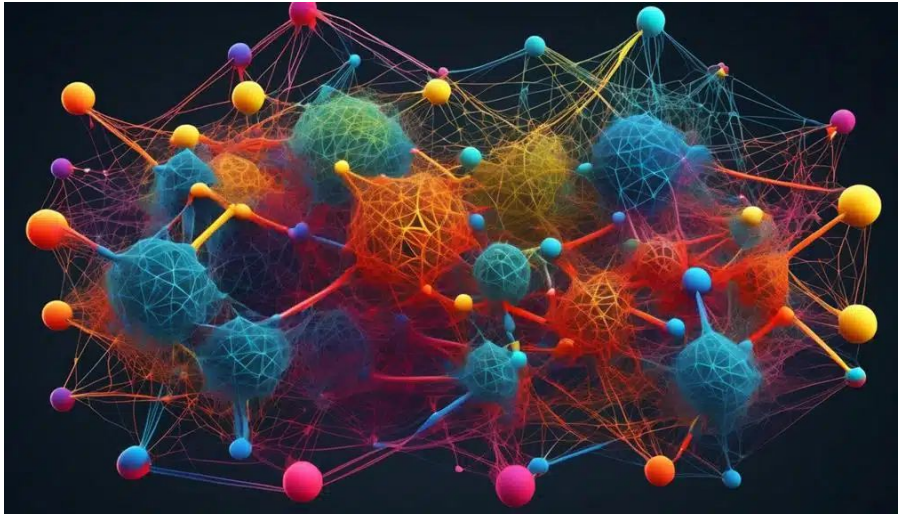
Backtracking

Camino Hamiltoniano: Código (cont.)

```
# grafo: grafo a recorrer
def camino_hamiltoniano(grafo):
  camino = []
  visitados = set()
  for v in grafo:
    if camino_hamiltoniano_dfs(grafo, v, visitados, camino):
      return camino
  return None
```

Control de Puntos de Articulación

Coloreo de Grafos



Créditos: blog.algorithmexamples.com

Coloreo de Grafos: Definición

- Tenemos un grafo $G=(V, E)$
- Queremos asignar un color a cada vértice v_i en V
- Dos vértices v_i, v_j adyacentes no pueden tener el mismo color

¿Es posible utilizar k colores o menos?

Coloreo de Grafos: Backtracking

- Si k=2, entonces estamos tratando de ver si el grafo es bipartito
  - Problema “fácil”: se resuelve con DFS o BFS
- Para k>=3, la cosa se complica muchísimo y rápidamente:
  - Para un grafo de n vértices, existiría un total de k^n posibles asignaciones de colores (muchas incompatibles)

Coloreo de Grafos: Ejemplo



Vamos a colorear las provincias argentinas, con $k=4$

Coloreo de Grafos: Ejemplo (cont.)

1. Si todas las provincias ya están coloreadas:
    a. Devuelvo True
2. Tomo la próxima provincia
3. Pruebo colorear con el próximo color
4. Si no es válido:
    a. Retrocedo y vuelvo a 3, para probar con otro color
5. Si es válido:
    a. Llamo recursivamente (vuelvo a 1) con la lista de provincias/colores actual
6. Si aún no tengo solución:
    a. Vuelvo a 3 para probar con otro color
7. Llegado a este punto, el problema no tiene solución

Coloreo de Grafos: Ejemplo (cont.)



1. Si todas las provincias ya están coloreadas:
   a. Devuelvo True
2. **Tomo la próxima provincia**
3. **Pruebo colorear con el próximo color**
4. Si no es válido:
   a. Retrocedo y vuelvo a 3, para probar con otro color
5. **Si es válido:**
   a. **Llamo recursivamente (vuelvo a 1) con la lista de provincias/colores actual**
6. Si aún no tengo solución:
   a. Vuelvo a 3 para probar con otro color
7. Llegado a este punto, el problema no tiene solución

Coloreo de Grafos: Ejemplo (cont.)



1. Si todas las provincias ya están coloreadas:
   a. Devuelvo True
2. **Tomo la próxima provincia**
3. **Pruebo colorear con el próximo color**
4. **Si no es válido:**
   a. **Retrocedo y vuelvo a 3, para probar con otro color**
5. Si es válido:
   a. Llamo recursivamente (vuelvo a 1) con la lista de provincias/colores actual
6. Si aún no tengo solución:
   a. Vuelvo a 3 para probar con otro color
7. Llegado a este punto, el problema no tiene solución

Coloreo de Grafos: Ejemplo (cont.)



1. Si todas las provincias ya están coloreadas:
   a. Devuelvo True
2. Tomo la próxima provincia
3. **Pruebo colorear con el próximo color**
4. Si no es válido:
   a. Retrocedo y vuelvo a 3, para probar con otro color
5. **Si es válido:**
   a. **Llamo recursivamente (vuelvo a 1) con la lista de provincias/colores actual**
6. Si aún no tengo solución:
   a. Vuelvo a 3 para probar con otro color
7. Llegado a este punto, el problema no tiene solución

Coloreo de Grafos: Ejemplo (cont.)



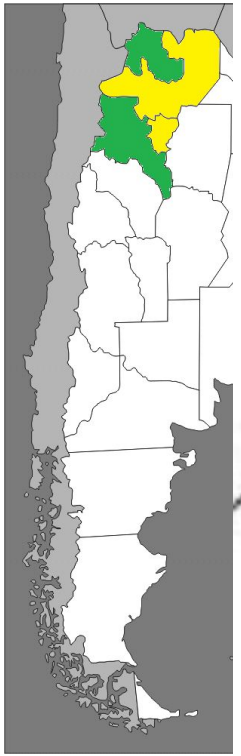
1. Si todas las provincias ya están coloreadas:
   a. Devuelvo True
2. **Tomo la próxima provincia**
3. **Pruebo colorear con el próximo color**
4. Si no es válido:
   a. Retrocedo y vuelvo a 3, para probar con otro color
5. **Si es válido:**
   a. **Llamo recursivamente (vuelvo a 1) con la lista de provincias/colores actual**
6. Si aún no tengo solución:
   a. Vuelvo a 3 para probar con otro color
7. Llegado a este punto, el problema no tiene solución

Coloreo de Grafos: Ejemplo (cont.)



1. Si todas las provincias ya están coloreadas:
   a. Devuelvo True
2. Tomo la próxima provincia
3. **Pruebo colorear con el próximo color**
4. **Si no es válido:**
   a. **Retrocedo y vuelvo a 3, para probar con otro color**
5. Si es válido:
   a. Llamo recursivamente (vuelvo a 1) con la lista de provincias/colores actual
6. Si aún no tengo solución:
   a. Vuelvo a 3 para probar con otro color
7. Llegado a este punto, el problema no tiene solución

Co

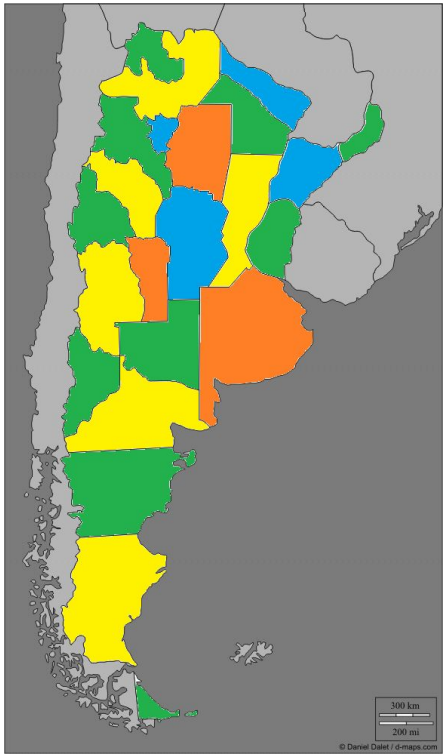




lor

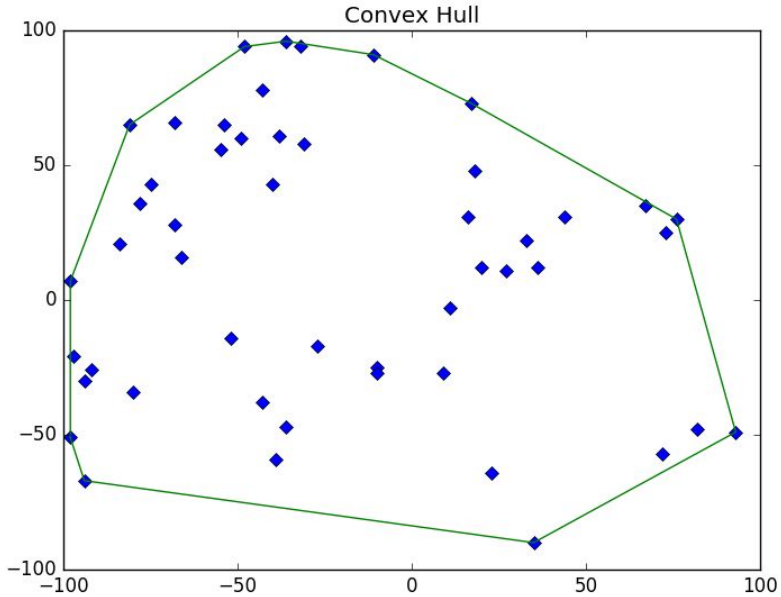
ción

Coloreo de Grafos: Ejemplo (cont.)



1. **Si todas las provincias ya están coloreadas:**
   a. **Devuelvo True**
2. Tomo la próxima provincia
3. Pruebo colorear con el próximo color
4. Si no es válido:
   a. Retrocedo y vuelvo a 3, para probar con otro color
5. Si es válido:
   a. Llamo recursivamente (vuelvo a 1) con la lista de provincias/colores actual
6. Si aún no tengo solución:
   a. Vuelvo a 3 para probar con otro color
7. Llegado a este punto, el problema no tiene solución

Casco Convexo



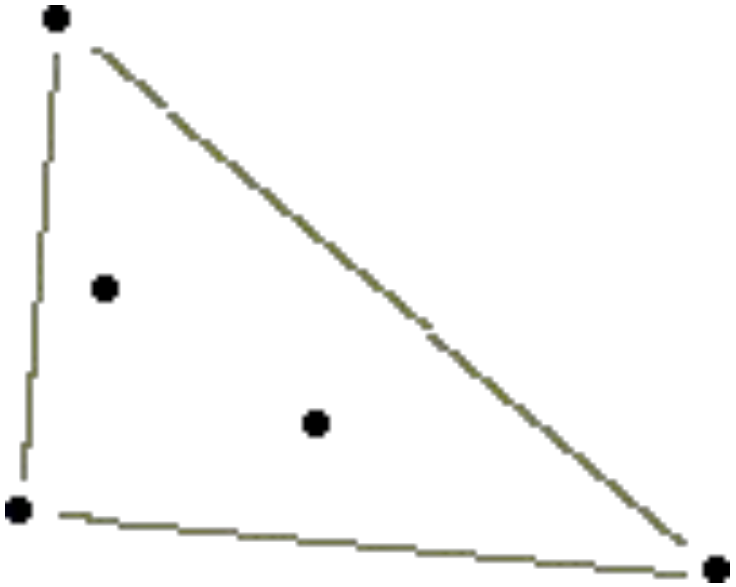
Créditos: dongtienvietnam.com

Casco Convexo: Definición

- Se tiene un conjunto $S=\{s_1, s_2, \ldots, s_n\}$ de puntos en el plano

El casco convexo es el polígono convexo más pequeño que incluye todos los puntos de S

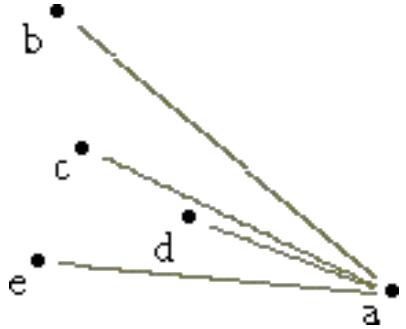
Casco Convexo: Definición (cont.)



Casco Convexo: Ejemplo

Preparación:

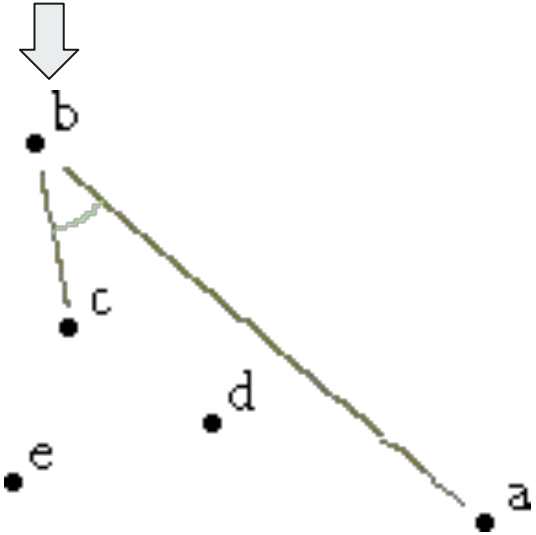
- Buscar el punto s_i con mayor coordenada x; si hay empate, tomar el de menor coordenada y
- Crear una lista de los puntos restantes ordenada crecientemente según el ángulo que forman con s_i; si hay empate, tomar el más cercano.



Casco Convexo: Ejemplo

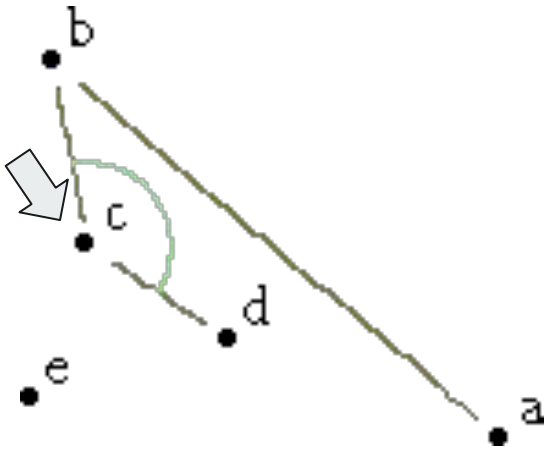
1. Si no quedan más puntos:
    a. Devuelvo el punto actual
2. Tomo el próximo punto
3. Calculo el ángulo que forma el punto actual, el punto seleccionado y el punto anterior
4. Si es mayor a 180º:
    a. Retrocedo y vuelvo a 2 para probar con otro punto
5. Si es menor o igual a 180º:
    a. Llamo recursivamente (vuelvo a 1) con el punto actual y el punto seleccionado
6. Si aún no tengo solución:
    a. vuelvo de la llamada recursiva descartando el punto actual

Casco Convexo: Ejemplo (cont.)



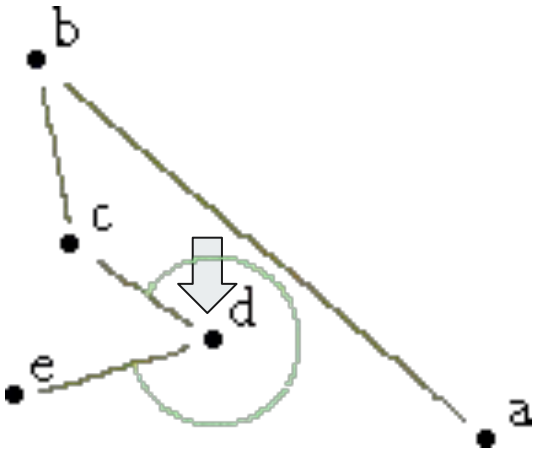
1. Si no quedan más puntos:
   a. Devuelvo el punto actual
2. **Tomo el próximo punto**
3. **Calculo el ángulo que forma el punto actual, el punto seleccionado y el punto anterior**
4. Si es mayor a 180°:
   a. Retrocedo y vuelvo a 2 para probar con otro punto
5. **Si es menor o igual a 180°:**
   a. **Llamo recursivamente (vuelvo a 1) con el punto actual y el punto seleccionado**
6. Si aún no tengo solución:
   a. vuelvo de la llamada recursiva descartando el punto actual

Casco Convexo: Ejemplo (cont.)



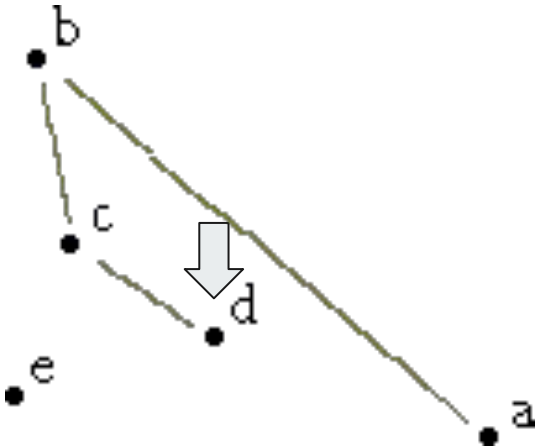
1. Si no quedan más puntos:
   a. Devuelvo el punto actual
2. **Tomo el próximo punto**
3. **Calculo el ángulo que forma el punto actual, el punto seleccionado y el punto anterior**
4. Si es mayor a 180°:
   a. Retrocedo y vuelvo a 2 para probar con otro punto
5. **Si es menor o igual a 180°:**
   a. **Llamo recursivamente (vuelvo a 1) con el punto actual y el punto seleccionado**
6. Si aún no tengo solución:
   a. vuelvo de la llamada recursiva descartando el punto actual

Casco Convexo: Ejemplo (cont.)



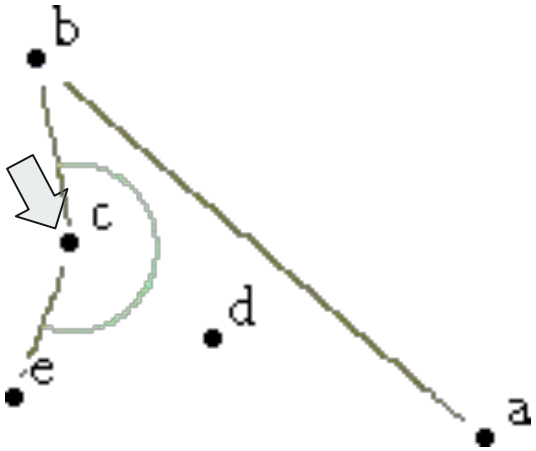
1. Si no quedan más puntos:
   a. Devuelvo el punto actual
2. **Tomo el próximo punto**
3. **Calculo el ángulo que forma el punto actual, el punto seleccionado y el punto anterior**
4. **Si es mayor a 180º:**
   a. **Retrocedo y vuelvo a 2 para probar con otro punto**
5. Si es menor o igual a 180º:
   a. Llamo recursivamente (vuelvo a 1) con el punto actual y el punto seleccionado
6. Si aún no tengo solución:
   a. vuelvo de la llamada recursiva descartando el punto actual

Casco Convexo: Ejemplo (cont.)



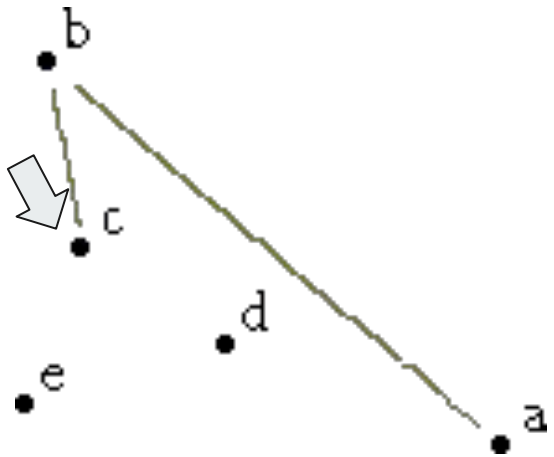
1. Si no quedan más puntos:
   a. Devuelvo el punto actual
2. Tomo el próximo punto
3. Calculo el ángulo que forma el punto actual, el punto seleccionado y el punto anterior
4. Si es mayor a 180º:
   a. Retrocedo y vuelvo a 2 para probar con otro punto
5. Si es menor o igual a 180º:
   a. Llamo recursivamente (vuelvo a 1) con el punto actual y el punto seleccionado
6. **Si aún no tengo solución:**
   a. **vuelvo de la llamada recursiva descartando el punto actual**

Casco Convexo: Ejemplo (cont.)



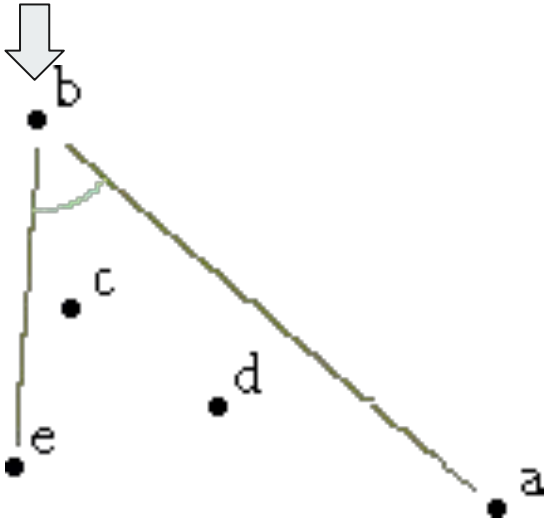
1. Si no quedan más puntos:
   a. Devuelvo el punto actual
2. **Tomo el próximo punto**
3. **Calculo el ángulo que forma el punto actual, el punto seleccionado y el punto anterior**
4. **Si es mayor a 180º:**
   a. **Retrocedo y vuelvo a 2 para probar con otro punto**
5. Si es menor o igual a 180º:
   a. Llamo recursivamente (vuelvo a 1) con el punto actual y el punto seleccionado
6. Si aún no tengo solución:
   a. vuelvo de la llamada recursiva descartando el punto actual

Casco Convexo: Ejemplo (cont.)



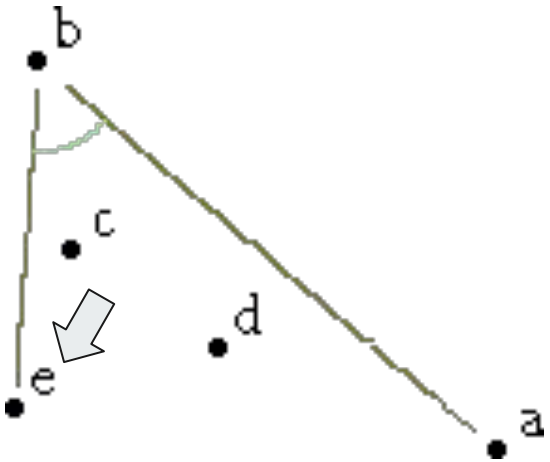
1. Si no quedan más puntos:
   a. Devuelvo el punto actual
2. Tomo el próximo punto
3. Calculo el ángulo que forma el punto actual, el punto seleccionado y el punto anterior
4. Si es mayor a 180º:
   a. Retrocedo y vuelvo a 2 para probar con otro punto
5. Si es menor o igual a 180º:
   a. Llamo recursivamente (vuelvo a 1) con el punto actual y el punto seleccionado
6. **Si aún no tengo solución:**
   a. **vuelvo de la llamada recursiva descartando el punto actual**

Casco Convexo: Ejemplo (cont.)



1. Si no quedan más puntos:
   a. Devuelvo el punto actual
2. **Tomo el próximo punto**
3. **Calculo el ángulo que forma el punto actual, el punto seleccionado y el punto anterior**
4. Si es mayor a 180°:
   a. Retrocedo y vuelvo a 2 para probar con otro punto
5. **Si es menor o igual a 180°:**
   a. **Llamo recursivamente (vuelvo a 1) con el punto actual y el punto seleccionado**
6. Si aún no tengo solución:
   a. vuelvo de la llamada recursiva descartando el punto actual

Casco Convexo: Ejemplo (cont.)



1. **Si no quedan más puntos:**
   a. **Devuelvo el punto actual**
2. Tomo el próximo punto
3. Calculo el ángulo que forma el punto actual, el punto seleccionado y el punto anterior
4. Si es mayor a 180º:
   a. Retrocedo y vuelvo a 2 para probar con otro punto
5. Si es menor o igual a 180º:
   a. Llamo recursivamente (vuelvo a 1) con el punto actual y el punto seleccionado
6. Si aún no tengo solución:
   a. vuelvo de la llamada recursiva descartando el punto actual

Casco Convexo: Otros algoritmos

Existen otros algoritmos para encontrar el casco convexo:

- Escaneo de Graham (Graham Scan)
- Marcha de Jarvis (Jarvis March)
- Algoritmo de Chan (Timothy Chan)
- Quickhull

Próxima clase

4	31/3/2026	**Práctica D&C, Greedy y Backtracking.**	
5	7/4/2026	Programación Dinámica I	K

D y C: Ejercicio #12 (Buchwald)

Greedy: Ejercicio #16 (Buchwald)

Backtracking Ejercicio #8 (Buchwald)



¡Muchas gracias!